

Project Work Data Mining

Marika Rucco

Settembre 2026

Contents

1	Introduzione e Motivazione	3
1.1	Differenze implementative rispetto al lavoro originale	3
1.1.1	Ambiente di esecuzione	3
1.1.2	Algoritmi utilizzati per il confronto	4
2	Dataset	5
2.1	Caratteristiche generali dei dataset	5
2.2	Mushroom	6
2.3	Retail	7
2.4	Pumsb	8
2.5	Kosarak	8
2.6	Chess	9
2.7	Connect	11
2.8	Confronto tra i dataset	12
3	Algoritmi	13
3.1	Cellular Learning Automata	13
3.1.1	Classi	13
3.1.2	Fase 1: individuazione degli itemset frequenti di dimensione 1	14
3.1.3	Fase 2: pruning e compressione del dataset	14
3.1.4	Fase 3: costruzione delle celle e delle liste di prossimità	14
3.1.5	Fase 4: pruning delle liste di prossimità e estrazione degli itemset frequenti	15
3.2	Apriori	16
3.2.1	Fase 1: individuazione degli itemset frequenti di dimensione 1	16
3.2.2	Fase 2: generazione dei candidati	16
3.2.3	Fase 3: verifica della proprietà di antimonotonicità	16
3.2.4	Fase 4: calcolo del supporto dei candidati ed estrazione degli itemset frequenti	16
3.3	FP-Growth	17
3.3.1	Fase 1: costruzione dell'FP-Tree	17

3.3.2	Fase 2: estrazione degli itemset frequenti	18
4	Analisi dei risultati	19
4.1	Dataset Mushroom	19
4.2	Dataset Retail	22
4.3	Dataset Pumsb	24
4.4	Dataset Kosarak	26
4.5	Dataset Chess	28
4.6	Dataset Connect	29
4.7	Confronto complessivo	31

1 Introduzione e Motivazione

L'obiettivo principale di questo progetto è l'implementazione e la valutazione sperimentale del metodo proposto da Sohrabi e Roshani nell'articolo Frequent Itemset Mining Using Cellular Learning Automata [1].

Il problema del frequent itemset mining consiste nell'individuare, all'interno di un insieme di transazioni, gli insiemi di item che ricorrono con una frequenza superiore a una determinata soglia, detta minimum support (minsup). Tale problema riveste particolare interesse nell'ambito dell'analisi dei dati e dell'estrazione di informazioni, soprattutto nel caso di dataset di grandi dimensioni.

Gli autori dell'articolo propongono un approccio basato sui Cellular Learning Automata (CLA), con l'obiettivo di affrontare il problema del frequent itemset mining attraverso un paradigma differente rispetto agli algoritmi tradizionali. In particolare, il metodo viene confrontato con algoritmi di riferimento, mostrando sperimentalmente risultati competitivi in termini di tempo di esecuzione.

In questo progetto viene implementato il metodo basato sui Cellular Learning Automata e ne vengono analizzate le prestazioni attraverso una serie di esperimenti. Per valutare il comportamento dell'approccio proposto, i risultati vengono confrontati con quelli ottenuti mediante due algoritmi alternativi per il frequent itemset mining. Gli esperimenti vengono inoltre eseguiti su dataset differenti e considerando diverse soglie di minsup, in modo da analizzare l'influenza di tali variabili sulle prestazioni degli algoritmi. Nelle sezioni successive verranno descritti i dataset utilizzati e le relative caratteristiche, verranno presentati i dettagli implementativi degli algoritmi considerati e verrà illustrato il protocollo sperimentale adottato. Infine, saranno discussi e confrontati i risultati ottenuti, con particolare attenzione ai tempi di esecuzione e al numero di frequent itemset individuati, per concludere con una valutazione complessiva degli approcci analizzati.

1.1 Differenze implementative rispetto al lavoro originale

L'implementazione del metodo basato sui Cellular Learning Automata (CLA) realizzata in questo progetto segue, per quanto possibile, le fasi descritte da Sohrabi e Roshani nel lavoro originale. Tuttavia, poiché l'articolo descrive il metodo da un punto di vista algoritmico e non fornisce un'implementazione direttamente riutilizzabile, alcune scelte relative alla rappresentazione dei dati, alla gestione delle strutture utilizzate, all'ambiente di esecuzione e agli algoritmi scelti per il confronto sono state definite autonomamente nel presente progetto.

1.1.1 Ambiente di esecuzione

Nell'articolo originale il metodo proposto e gli algoritmi utilizzati per il confronto sono stati eseguiti nell'ambiente Visual Studio .NET 2013, utilizzando il linguaggio C# su un computer equipaggiato con sistema operativo Windows 7 x64, processore Intel Core 2 Quad a 2.8 GHz e 8 GB di RAM. Nel presente

progetto, invece, tutte le implementazioni sono state sviluppate in Python ed eseguite nell'ambiente Google Colab, utilizzando la CPU messa a disposizione dalla piattaforma. Di conseguenza, i tempi di esecuzione riportati nella relazione non sono direttamente confrontabili in valore assoluto con quelli presentati nell'articolo originale, e il confronto effettuato in questo progetto deve essere interpretato principalmente in termini relativi, tra le tre implementazioni eseguite nelle medesime condizioni sperimentali.

1.1.2 Algoritmi utilizzati per il confronto

Anche la scelta degli algoritmi di confronto differisce, in parte, da quella adottata nel lavoro originale. Nella sezione sperimentale dell'articolo, gli autori confrontano il metodo CLA con SABMA (Sohrabi & Barforoush, 2013) e PP-Tree (Javed & Khokhar, 2004), entrambi algoritmi progettati per sfruttare forme di elaborazione parallela. Gli stessi autori dichiarano tuttavia, nella parte introduttiva dell'articolo, di aver confrontato il metodo proposto anche con Apriori, FP-Growth e BitTable; nella sezione dedicata ai risultati sperimentali, però, non vengono riportati dati relativi a questo confronto, che rimane quindi privo di un riscontro numerico verificabile all'interno del paper.

Nel presente progetto sono stati scelti Apriori e FP-Growth come algoritmi di riferimento, questa scelta è motivata da due ragioni. In primo luogo, si tratta degli stessi due algoritmi che gli autori dichiarano di aver utilizzato come baseline, il che rende il confronto proposto in questa relazione coerente con l'impostazione sperimentale del lavoro originale, pur senza poterne verificare i risultati numerici corrispondenti. In secondo luogo, a differenza di SABMA e PP-Tree, Apriori e FP-Growth non richiedono un'infrastruttura per il calcolo parallelo e sono stati quindi implementati direttamente in Python all'interno dello stesso notebook utilizzato per CLA, garantendo un confronto omogeneo in termini di linguaggio, piattaforma e condizioni di esecuzione. La riproduzione del confronto con SABMA e PP-Tree avrebbe invece richiesto un ambiente hardware e software specificamente configurato per supportare le architetture parallele previste da tali approcci, non disponibile per questo progetto.

Pertanto, il confronto proposto nella presente relazione non ha l'obiettivo di replicare gli esperimenti dell'articolo originale, per i quali mancano peraltro risultati completi relativi ad Apriori e FP-Growth, ma di fornire una valutazione sperimentale autonoma e verificabile del metodo CLA rispetto a due algoritmi di riferimento comunemente utilizzati in letteratura, implementati e testati nelle medesime condizioni. L'utilizzo della stessa piattaforma per tutte e tre le implementazioni, oltre agli stessi dataset e agli stessi valori di minsup, consente di concentrarsi sulle differenze relative nei tempi di esecuzione e nel numero di itemset estratti.

2 Dataset

Per la valutazione sperimentale degli algoritmi sono stati utilizzati quattro dataset utilizzati anche nella sezione "Experimental results" dell'articolo di riferimento: Mushroom, Retail, Pumsb e Kosarak e due dataset aggiuntivi: Chess e Connect.

Tutti i dataset sono disponibili presso la Frequent Itemset Mining Dataset Repository (FIMI), una repository dedicata alla raccolta di dataset utilizzati per la valutazione di algoritmi di frequent itemset mining. In particolare, la repository FIMI contiene versioni dei dataset già preprocessate e rappresentate in forma transazionale, ovvero come sequenze di item appartenenti a ciascuna transazione.

La scelta di questi dataset è motivata anche dalla necessità di rendere il confronto con il lavoro di Sohrabi e Roshani il più possibile significativo. Gli stessi dataset appartengono infatti all'insieme di benchmark comunemente utilizzati nella letteratura sul frequent itemset mining e sono rappresentativi di caratteristiche differenti in termini di numero di transazioni, numero di item e lunghezza media delle transazioni. Il lavoro di Sohrabi e Roshani valuta il metodo basato sui Cellular Learning Automata considerando differenti valori della soglia di supporto. L'utilizzo degli stessi dataset permette quindi di effettuare un confronto sperimentale coerente con il contesto di riferimento del metodo implementato.

Un aspetto importante riguarda la rappresentazione dei dati. I file messi a disposizione dalla repository FIMI non sono necessariamente accompagnati dalle informazioni semantiche relative alle variabili originarie: il preprocessing effettuato per il frequent itemset mining rappresenta direttamente ogni transazione come un insieme di item. Di conseguenza, le informazioni sul dominio applicativo dei dataset vengono descritte separatamente, quando disponibili, mentre gli algoritmi considerano esclusivamente la rappresentazione transazionale.

2.1 Caratteristiche generali dei dataset

I dataset presentano caratteristiche significativamente differenti. In particolare, Mushroom, Pumsb, Chess e Connect sono caratterizzati da transazioni relativamente lunghe e da una densità maggiore rispetto a Retail e Kosarak, che presentano invece un numero elevato di item distinti e transazioni mediamente più brevi. Questa varietà rende i dataset adatti a valutare il comportamento degli algoritmi in condizioni differenti. La tabella seguente riassume le principali caratteristiche dei dataset utilizzati. Il numero di item rappresenta il numero di item distinti presenti nel dataset, mentre la lunghezza media della transazione indica il numero medio di item contenuti in una singola transazione.

Dataset	Transazioni	Item	Lunghezza media	Densità
Mushroom	8124	119	23	19,33%
Retail	88162	16470	10	0,06%
Pumsb	49046	2113	74	3,50%
Kosarak	990002	41270	8	0,02%
Chess	3196	75	37	49,33%
Connect	67557	129	43	33,33%

Table 1: Principali caratteristiche dei dataset utilizzati

La densità può essere interpretata come il rapporto tra il numero medio di item presenti in una transazione e il numero totale di item distinti nel dataset. Un valore elevato indica quindi che le transazioni contengono, mediamente, una porzione maggiore dell'insieme complessivo degli item. Al contrario, nei dataset sparsi una singola transazione contiene generalmente solo una piccola parte degli item disponibili.

2.2 Mushroom

Il dataset Mushroom è caratterizzato da dati relativi ai funghi e nella versione presente nella repository FIMI, è stato trasformato in una rappresentazione adatta al frequent itemset mining. Il dataset contiene 8124 transazioni e 119 item distinti, con una lunghezza media di circa 23 item per transazione. Mushroom permette di fornire una descrizione dettagliata delle variabili originarie, poiché i dati di partenza sono osservazioni descritte attraverso caratteristiche categoriche dei funghi. Le variabili presenti nel dataset originale sono:

Variabile	Descrizione
cap-shape	Forma del cappello
cap-surface	Superficie del cappello
cap-color	Colore del cappello
bruises	Presenza di ammaccature
odor	Odore
gill-attachment	Tipo di attacco delle lamelle
gill-spacing	Spaziatura delle lamelle
gill-size	Dimensione delle lamelle
gill-color	Colore delle lamelle
stalk-shape	Forma del gambo
stalk-root	Tipo di radice del gambo
stalk-surface-above-ring	Superficie del gambo sopra l'anello
stalk-surface-below-ring	Superficie del gambo sotto l'anello
stalk-color-above-ring	Colore del gambo sopra l'anello
stalk-color-below-ring	Colore del gambo sotto l'anello
veil-type	Tipo di velo
veil-color	Colore del velo
ring-number	Numero di anelli
ring-type	Tipo di anello
spore-print-color	Colore della sporata
population	Distribuzione della popolazione
habitat	Habitat

Table 2: Variabili originarie del dataset Mushroom

Nella rappresentazione utilizzata dagli algoritmi, le variabili categoriche vengono rappresentate attraverso coppie attributo-valore. Ad esempio, una caratteristica come `cap-shape=x` viene considerata come un singolo item. Una transazione è quindi costituita dall'insieme degli item corrispondenti alle caratteristiche osservate in una determinata istanza. Questa trasformazione permette di passare dalla rappresentazione originaria del dataset a quella richiesta dagli algoritmi di frequent itemset mining, nei quali ogni transazione è considerata come un insieme di item.

2.3 Retail

Il dataset Retail contiene dati relativi alle transazioni di un supermercato belga anonimo. I dati sono stati raccolti in tre periodi non consecutivi, compresi complessivamente in circa cinque mesi tra la fine del 1999 e il 2000. Il dataset originale contiene 88163 ricevute e 16470 SKU (Stock Keeping Unit) distinti. Nel dataset originale ogni record contiene informazioni relative alla transazione e all'acquisto effettuato. In particolare, sono presenti:

Variabile	Descrizione
date	Data di acquisto
receipt_nr	Identificativo della ricevuta
article_nr	Identificativo dell'articolo
amount	Quantità acquistata
price	Prezzo dell'articolo
customer_nr	Identificativo del cliente

Table 3: Variabili originarie del dataset Retail

La versione utilizzata per il frequent itemset mining non mantiene direttamente tutte queste informazioni, ma rappresenta i dati in forma transazionale. Gli item corrispondono quindi agli articoli acquistati, mentre una transazione rappresenta l'insieme degli articoli associati a una determinata ricevuta. Il dataset Retail presenta una struttura particolarmente sparsa: a fronte di 16470 item distinti, una transazione contiene mediamente circa 10 item. Questa caratteristica permette di analizzare il comportamento degli algoritmi in presenza di un elevato numero di item disponibili e di transazioni relativamente brevi.

2.4 Pumsb

Il dataset Pumsb deriva da dati relativi alla popolazione e alle abitazioni (population and housing census data). La versione disponibile nella repository FIMI è già preprocessata e rappresentata in forma transazionale. Il dataset contiene 49046 transazioni e 2113 item distinti, con una lunghezza media di circa 74 item per transazione. Rispetto a Retail e Kosarak, il dataset presenta transazioni significativamente più lunghe e una maggiore densità. A differenza del dataset Mushroom, nella versione transazionale utilizzata per gli esperimenti gli item sono rappresentati mediante identificativi e non è disponibile, nella repository FIMI, una corrispondenza diretta tra ciascun identificativo e una specifica variabile semantica del dataset originale. Per questo motivo, nell'implementazione gli item di Pumsb vengono trattati esclusivamente come elementi della transazione, senza attribuire loro un significato applicativo specifico.

2.5 Kosarak

Il dataset Kosarak è costituito da dati di tipo click-stream anonimizzati provenienti da un portale di news online ungherese. La repository FIMI identifica il dataset come un insieme di dati transazionali ottenuti da tali click-stream. Il dataset contiene circa 990002 transazioni e oltre 41000 item distinti, con una lunghezza media di circa 8 item per transazione. La combinazione tra il numero molto elevato di transazioni, l'elevato numero di item distinti e la ridotta lunghezza media delle transazioni rende Kosarak uno dei dataset più sparsi tra quelli considerati. Gli item presenti nel dataset sono anonimizzati e non possiedono quindi, nella versione distribuita dalla repository FIMI, un'interpretazione semantica direttamente utilizzabile. Di conseguenza, ogni

identificativo viene considerato esclusivamente come un item e l'analisi si concentra sulle relazioni di frequenza tra gli item presenti nelle diverse transazioni.

2.6 Chess

Il dataset Chess deriva dal dataset "Chess End-Game – King+Rook versus King+Pawn on a7", originariamente disponibile presso la UCI Machine Learning Repository. Il dataset originale descrive posizioni di una specifica situazione di finale di partita a scacchi e contiene 3196 istanze. Ogni istanza è rappresentata mediante 36 attributi discreti che descrivono diverse proprietà della posizione sulla scacchiera, oltre a una variabile che indica se il bianco può vincere (won) oppure non può vincere (nowin). Gli attributi descrittivi presenti nel dataset originale sono:

Variabile	Descrizione
bkbk	Posizione del re nero rispetto alla casella b1
bknwy	Relazione tra il re nero e la posizione nwy
bkon8	Relazione del re nero con la casella 8
bkona	Relazione del re nero con la posizione a
bkspr	Proprietà relativa al re nero e al pedone
bkxbq	Relazione del re nero con il pezzo bianco sulla diagonale
bkxcr	Relazione del re nero con la torre
bkwpx	Relazione del re nero con il pedone bianco
blxwp	Relazione tra il pedone nero e il pedone bianco
bxqsq	Relazione tra il pedone nero e la regina
cntxt	Contesto della posizione
dsopp	Posizione relativa dei pedoni
dwipd	Proprietà relativa alla struttura dei pedoni
hdchk	Presenza di scacco
katri	Configurazione relativa dei pezzi
mulch	Configurazione multipla dei pezzi
qxmsq	Relazione della regina con la posizione
r2ar8	Relazione della torre con le caselle della scacchiera
reskd	Posizione relativa del re rispetto al pedone
reskr	Posizione relativa dei re
rimmx	Distanza o relazione tra torre e re
rkxwp	Relazione tra torre e pedone bianco
rxmsq	Relazione della torre con la posizione
simpl	Livello di semplicità della posizione
skach	Proprietà relativa allo scacco
skewr	Relazione tra re e torre
skrxp	Relazione tra re, torre e pedone
spcop	Configurazione spaziale dei pezzi
stlmt	Condizione di stallo
thrsk	Presenza di una minaccia al re
wkcti	Posizione del re bianco rispetto alla struttura
wkna8	Relazione del re bianco con la casella a8
wknck	Relazione del re bianco con il re nero
wkovl	Posizione relativa del re bianco
wkpos	Posizione del re bianco
wtoeg	Relazione del re bianco con la configurazione di gioco

Table 4: Variabili originarie del dataset Chess

I nomi degli attributi del dataset originale sono abbreviati e fanno riferimento a proprietà specifiche della configurazione di gioco; la documentazione originale non fornisce quindi descrizioni intuitive paragonabili, ad esempio, alle variabili del dataset Mushroom. Per questo motivo è preferibile non attribuire agli attributi un significato semantico più specifico di quello effettivamente documentato nella fonte originale. Il dataset contiene esclusivamente variabili dis-

crete e non presenta valori mancanti.

La versione presente nella repository FIMI non conserva tuttavia questa rappresentazione originaria.

2.7 Connect

Il dataset Connect deriva dal dataset Connect-4 della UCI Machine Learning Repository. Esso contiene le posizioni legali del gioco Connect-4, corrispondenti a posizioni raggiunte dopo otto semimosse (8-ply) nelle quali nessuno dei due giocatori ha ancora vinto e la mossa successiva non è forzata. Il dataset originale contiene 42 variabili categoriche, una per ciascuna delle 42 celle della griglia di gioco, e una variabile che rappresenta il valore teorico della posizione per il primo giocatore.

Le 42 variabili sono organizzate secondo le coordinate della griglia 6×7 :

Colonna	Variabili	Valori possibili
a	a1, a2, a3, a4, a5, a6	x, o, b
b	b1, b2, b3, b4, b5, b6	x, o, b
c	c1, c2, c3, c4, c5, c6	x, o, b
d	d1, d2, d3, d4, d5, d6	x, o, b
e	e1, e2, e3, e4, e5, e6	x, o, b
f	f1, f2, f3, f4, f5, f6	x, o, b
g	g1, g2, g3, g4, g5, g6	x, o, b

Table 5: Variabili originarie del dataset Connect

Il simbolo x indica una cella occupata dal primo giocatore, o una cella occupata dal secondo giocatore e b una cella vuota. La disposizione delle variabili corrisponde alle celle della scacchiera, con le colonne indicate dalle lettere da a a g e le righe numerate da 1 a 6. La variabile aggiuntiva class rappresenta invece la classe della posizione e può assumere i valori win, loss o draw.

Anche in questo caso, la versione utilizzata per il frequent itemset mining non mantiene direttamente la struttura originale composta da 42 variabili categoriche. Il preprocessing converte le configurazioni delle celle in item e rappresenta ogni posizione come una transazione. Nella repository FIMI il dataset contiene 67557 transazioni e 129 item distinti, con una lunghezza media di circa 43 item e una densità pari a circa 33,33

La differenza tra le 42 variabili originali e i 129 item della versione FIMI è dovuta al fatto che una variabile categorica può dare origine a più item distinti. Ad esempio, la variabile a1, che nel dataset originale può assumere i valori x, o oppure b, può essere rappresentata nella forma transazionale attraverso item distinti corrispondenti alle diverse configurazioni attributo-valore. In questo modo ogni posizione del gioco viene trasformata in un insieme di item e può essere elaborata direttamente dagli algoritmi di frequent itemset mining.

2.8 Confronto tra i dataset

La scelta dei sei dataset consente di valutare gli algoritmi in scenari caratterizzati da proprietà significativamente differenti. In particolare, i dataset considerati coprono un ampio intervallo sia per quanto riguarda il numero di transazioni e di item distinti, sia per la lunghezza media delle transazioni e la densità della rappresentazione. Questa varietà permette di analizzare il comportamento degli algoritmi di frequent itemset mining in presenza di dataset con caratteristiche tra loro molto diverse.

Il dataset Mushroom presenta un numero relativamente contenuto di item distinti e transazioni di lunghezza moderata, con una densità significativamente maggiore rispetto ai dataset Retail, Pumsb e Kosarak. Pumsb si distingue invece per la lunghezza media delle transazioni, pari a circa 74 item, a fronte di un numero di item distinti pari a 2113. Questo determina una struttura caratterizzata da transazioni relativamente lunghe, pur mantenendo una densità inferiore rispetto a Mushroom, Chess e Connect.

I dataset Retail e Kosarak rappresentano invece scenari caratterizzati da una maggiore sparsità degli item. Retail contiene oltre 16000 item distinti, mentre Kosarak supera i 41000 item. In entrambi i casi, la lunghezza media delle transazioni è relativamente ridotta, pari rispettivamente a circa 10 e 8 item. La combinazione tra un elevato numero di item disponibili e transazioni brevi determina una struttura fortemente sparsa, nella quale ciascuna transazione contiene soltanto una piccola frazione dell'insieme complessivo degli item.

I dataset Chess e Connect introducono una caratteristica differente rispetto ai precedenti. Entrambi presentano un numero relativamente contenuto di item distinti, pari rispettivamente a 75 e 129, ma sono caratterizzati da una densità elevata. Chess presenta una lunghezza media di circa 37 item per transazione e una densità prossima al 50%, mentre Connect presenta circa 43 item per transazione e una densità superiore al 30%. Queste proprietà rendono i due dataset particolarmente differenti rispetto a Retail e Kosarak: nonostante il numero ridotto di item distinti, ogni transazione contiene una porzione significativa dell'insieme complessivo degli item.

Nel complesso, i dataset possono quindi essere considerati rappresentativi di diversi scenari di frequent itemset mining. Mushroom, Chess e Connect costituiscono dataset relativamente densi, sebbene con differenti numeri di item e lunghezze delle transazioni. Pumsb rappresenta invece un caso caratterizzato principalmente da transazioni molto lunghe e da un numero intermedio di item distinti. Retail e Kosarak rappresentano infine scenari molto sparsi, con un numero molto elevato di item distinti e transazioni mediamente brevi.

Queste differenze sono particolarmente rilevanti nel contesto del frequent itemset mining, poiché il numero di item distinti, la lunghezza delle transazioni e la densità del dataset possono influenzare significativamente sia il numero di itemset candidati sia il numero di itemset frequenti individuati. Nei dataset densi, la presenza di un numero elevato di item all'interno delle singole transazioni può favorire la formazione di combinazioni frequenti e aumentare lo spazio di ricerca esplorato dagli algoritmi. Nei dataset sparsi, invece, la maggiore varietà

degli item e la ridotta sovrapposizione tra le transazioni possono ridurre la frequenza delle combinazioni di item, pur introducendo un numero molto elevato di possibili itemset.

La combinazione dei sei dataset permette pertanto di valutare gli algoritmi non soltanto in funzione delle loro prestazioni su dataset di dimensioni differenti, ma anche rispetto a differenti strutture di distribuzione degli item. In particolare, il confronto tra dataset densi e sparsi consente di osservare come le caratteristiche della rappresentazione transazionale possano incidere sul costo computazionale della ricerca degli itemset frequenti e sul comportamento degli algoritmi al variare della soglia minima di supporto.

3 Algoritmi

In questa sezione vengono descritti gli algoritmi implementati per il problema di frequent itemset mining. L'implementazione è stata realizzata seguendo, per quanto possibile, la struttura e i passaggi descritti nell'articolo di riferimento per il metodo basato sui Cellular Learning Automata e implementando, inoltre, gli algoritmi Apriori e FP-Growth come metodo di paragone. Gli algoritmi ricevono in input un dataset rappresentato in forma transazionale, una soglia minima di supporto e una lunghezza massima degli itemset da individuare. L'obiettivo è ottenere l'insieme degli itemset frequenti, ovvero degli itemset il cui supporto risulta maggiore o uguale alla soglia specificata.

3.1 Cellular Learning Automata

Il primo algoritmo implementato è il metodo basato sui Cellular Learning Automata (CLA), proposto da Sohrabi e Roshani. L'idea alla base del metodo consiste nel rappresentare le relazioni tra gli item attraverso una struttura di celle e liste di prossimità, evitando di generare esplicitamente tutti i candidati possibili come avviene nell'algoritmo Apriori. L'implementazione è organizzata nelle seguenti quattro fasi:

- individuazione degli itemset frequenti di dimensione 1;
- pruning e compressione del dataset;
- costruzione delle celle e delle relative liste di prossimità;
- pruning delle liste di prossimità estrazione degli itemset frequenti.

3.1.1 Classi

Come prima parte dell'implementazione del metodo proposto nell'articolo sono state definite quattro classi:

- Item: rappresenta un singolo item e contiene il nome dell'item e il relativo supporto;

- Transaction: rappresenta una transazione del dataset. Contiene la lista degli item presenti e un attributo `iteration_number`, utilizzato per indicare quante volte la transazione si ripete dopo la fase di compressione;
- Cell: rappresenta una cella dell'automa ed è caratterizzata dal nome dell'item associato e dalla relativa `proximity_list`;
- Neighbor: rappresenta un elemento presente all'interno di una lista di prossimità. Oltre al nome e all'`iteration_number`, un Neighbor può contenere a sua volta una `proximity_list`, permettendo di rappresentare relazioni tra più livelli di item.

3.1.2 Fase 1: individuazione degli itemset frequenti di dimensione 1

La prima fase è implementata dalla funzione `find_1_itemsets(dataset, minsup)`. La funzione inizialmente costruisce un dizionario nel quale, per ogni item presente nel dataset, viene memorizzato il relativo supporto. Il dataset viene quindi scansionato e il conteggio associato a ciascun item viene incrementato ogni volta che l'item viene incontrato. Terminata la scansione, vengono selezionati solamente gli item il cui supporto è maggiore o uguale a `minsup`. Per ciascuno di essi viene creato un oggetto `Item`, contenente il nome dell'item e il relativo supporto. Gli itemset ottenuti vengono infine ordinati alfabeticamente e restituiti dalla funzione.

3.1.3 Fase 2: pruning e compressione del dataset

La funzione `prune_and_compress_dataset(dataset, frequent_itemset)` implementa la seconda fase dell'algoritmo. Lo scopo di questa fase è ridurre la quantità di dati utilizzata nelle operazioni successive. Per prima cosa viene costruito un insieme contenente i nomi degli 1-itemset frequenti. Successivamente, ogni transazione viene analizzata e vengono mantenuti solamente gli item appartenenti a tale insieme. Le transazioni risultanti vengono ordinate e utilizzate come chiavi di un dizionario. In questo modo, due transazioni che dopo il pruning contengono gli stessi item vengono considerate equivalenti anche se originariamente gli item erano presenti in un ordine differente. Quando una stessa transazione compare più volte, invece di memorizzarla nuovamente viene incrementato il relativo `iteration_number`. La funzione restituisce quindi una lista di transazioni compresse, nella quale ogni oggetto rappresenta una particolare combinazione di item e il numero di occorrenze nel dataset originale.

3.1.4 Fase 3: costruzione delle celle e delle liste di prossimità

Le funzioni `run_cellular_automata(compressed_dataset, frequent_itemset)` e `update_proximity_list(neighbor, remaining_items, iteration_number)` implementano la fase centrale del metodo CLA. La prima funzione crea una cella per ciascun itemset frequente di dimensione 1. Ogni cella è quindi identificata dal nome dell'item corrispondente. Per ogni transazione del dataset compresso

viene costruita una lista ordinata degli item frequenti presenti nella transazione. L'ordinamento permette di mantenere una rappresentazione coerente delle relazioni tra gli item. Per ciascuna cella presente nella transazione vengono quindi considerati gli altri item della transazione e queste informazioni vengono utilizzate per aggiornare la relativa lista di prossimità. L'aggiornamento delle liste di prossimità viene effettuato dalla seconda funzione. Per ogni item associato alla cella viene creato, se necessario, un oggetto Neighbor e viene incrementato il relativo `iteration_number` in base al numero di occorrenze della transazione compressa. La funzione considera inoltre gli item successivi nella lista e memorizza le relazioni tra coppie di elementi all'interno della lista di prossimità del Neighbor. Questa struttura ricorsiva permette di rappresentare non solo la presenza di singoli item insieme alla cella, ma anche relazioni tra item a livelli successivi.

3.1.5 Fase 4: pruning delle liste di prossimità e estrazione degli itemset frequenti

Una volta costruite le celle, viene eseguita una nuova fase di pruning attraverso le funzioni `prune_cells(cells, minsup)` e `prune_proximity_list(neighbor, minsup)`. La seconda funzione analizza i Neighbor presenti in una lista di prossimità e mantiene solamente quelli il cui `iteration_number` è maggiore o uguale a `minsup`. Per ogni elemento mantenuto, la funzione viene richiamata ricorsivamente sulla relativa lista di prossimità. In questo modo il pruning viene applicato a tutti i livelli della struttura, eliminando le relazioni che non soddisfano il supporto minimo richiesto. La prima funzione applica questo procedimento alla lista di prossimità di ogni cella. Dopo il pruning, gli itemset vengono estratti attraverso le funzioni `scan_cells(cells, max_len)` e `collect_itemsets(neighbor, prefix_item, max_len)`. La funzione `scan_cells()` considera ogni cella come punto di partenza. L'item associato alla cella costituisce un itemset frequente di dimensione 1 e viene successivamente utilizzata la funzione `collect_itemsets()` per costruire itemset di dimensione maggiore. La funzione procede ricorsivamente attraverso le liste di prossimità. Ad ogni livello viene aggiunto al prefisso corrente il nome del Neighbor considerato. Se la lunghezza dell'itemset non ha ancora raggiunto il valore massimo `max_len`, la funzione continua ricorsivamente l'esplorazione della relativa lista di prossimità. Poiché lo stesso itemset può essere raggiunto partendo da celle differenti o attraverso ordini differenti degli item, è possibile ottenere duplicati. La funzione `remove_duplicate(itemsets)` risolve questo problema ordinando gli elementi di ciascun itemset, trasformandolo in una tupla e inserendolo in un insieme. In questo modo itemset costituiti dagli stessi elementi vengono considerati equivalenti indipendentemente dal loro ordine. L'intero procedimento viene coordinato dalla funzione `cla_mining(dataset, minsup, max_len)`, che richiama in sequenza tutte le fasi precedentemente descritte e restituisce la lista finale degli itemset individuati.

3.2 Apriori

Il secondo algoritmo implementato è Apriori, utilizzato come metodo di confronto con il metodo basato sui Cellular Learning Automata. A differenza del metodo CLA, Apriori procede attraverso una generazione iterativa di candidati. A partire dagli itemset frequenti di dimensione 1 vengono generati gli itemset candidati di dimensione successiva. I candidati vengono quindi sottoposti a un controllo basato sulla proprietà di antimonotonicità e, successivamente, viene calcolato il loro supporto attraverso una scansione del dataset.

3.2.1 Fase 1: individuazione degli itemset frequenti di dimensione 1

La funzione `find_1_itemsets_apriori(dataset, minsup)` rappresenta il punto di partenza dell'algoritmo. La funzione scansiona tutte le transazioni e costruisce un dizionario contenente il supporto di ciascun item. Al termine della scansione vengono mantenuti solamente gli item il cui supporto è maggiore o uguale a `minsup`. Gli item frequenti vengono rappresentati come tuple contenenti un singolo elemento e vengono infine ordinati.

3.2.2 Fase 2: generazione dei candidati

La funzione `generate_candidate(frequent_itemsets, target_length)` genera gli itemset candidati. In particolare, la funzione riceve gli itemset frequenti individuati al livello precedente e genera candidati della dimensione richiesta. Gli itemset vengono ordinati e vengono confrontate le coppie di itemset che condividono lo stesso prefisso. Quando la condizione è soddisfatta, i due itemset vengono uniti utilizzando l'unione tra insiemi. Il candidato risultante viene ordinato e trasformato in una tupla. L'utilizzo di un set per memorizzare i candidati permette di evitare duplicati durante la generazione.

3.2.3 Fase 3: verifica della proprietà di antimonotonicità

Dopo la generazione, i candidati vengono filtrati attraverso la funzione `verify_antimonotonicity(candidate_itemset, previous_itemsets)`. La funzione verifica che tutti i sottoinsiemi ottenuti eliminando un singolo elemento dal candidato siano presenti nell'insieme degli itemset frequenti del livello precedente. Se almeno uno di questi sottoinsiemi non è frequente, il candidato viene eliminato. Questa fase sfrutta quindi la proprietà di antimonotonicità: se un itemset è frequente, anche tutti i suoi sottoinsiemi devono essere frequenti. Di conseguenza, la presenza di un sottoinsieme non frequente permette di escludere direttamente il candidato senza doverne calcolare il supporto.

3.2.4 Fase 4: calcolo del supporto dei candidati ed estrazione degli itemset frequenti

La funzione `count_candidates_support(dataset, candidate_itemsets)` calcola il supporto dei candidati rimasti dopo la verifica della proprietà di antimonotonicità.

tonicità. Per ogni transazione, gli item vengono convertiti in un insieme. Successivamente, per ciascun candidato viene verificato se l'insieme dei suoi elementi è un sottoinsieme della transazione attraverso l'operazione `issubset()`. Se il candidato è contenuto nella transazione, il relativo supporto viene incrementato. Al termine della scansione viene restituito un dizionario contenente il supporto di ciascun candidato. La funzione `filter_frequent_itemsets(candidate_support, minsup)` riceve i supporti calcolati per i candidati e mantiene solamente gli itemset che soddisfano la soglia minima di supporto. Gli itemset selezionati vengono ordinati e utilizzati come base per la generazione dei candidati del livello successivo. La funzione `apriori_mining(dataset, minsup, max_len)` coordina tutte le fasi dell'algoritmo Apriori. Dopo aver individuato gli itemset frequenti di dimensione 1, la funzione procede iterativamente aumentando la dimensione degli itemset fino a `max_len`. Ad ogni iterazione vengono generati i candidati, viene verificata la proprietà di antimonotonicità, viene calcolato il supporto dei candidati rimasti e vengono selezionati gli itemset frequenti. Se a un determinato livello non vengono individuati itemset frequenti, l'esecuzione viene interrotta anticipatamente. Infine, gli itemset frequenti individuati ai diversi livelli vengono raccolti in un'unica lista che costituisce il risultato dell'algoritmo.

3.3 FP-Growth

Il terzo algoritmo implementato per il confronto con il metodo basato sui Cellular Learning Automata è FP-Growth. Questo algoritmo permette di individuare gli itemset frequenti evitando la generazione esplicita dell'insieme dei candidati, sfruttando una struttura dati compatta denominata FP-Tree (Frequent Pattern Tree). L'implementazione è suddivisa in due fasi principali:

1. costruzione dell'FP-Tree a partire dalle transazioni del dataset;
2. estrazione ricorsiva degli itemset frequenti a partire dall'FP-Tree.

3.3.1 Fase 1: costruzione dell'FP-Tree

La prima fase consiste nella costruzione dell'FP-Tree. Per rappresentare i nodi dell'albero viene definita la classe `FPTreeNode`. Ogni nodo rappresenta un item e contiene le informazioni necessarie per descrivere la struttura gerarchica dell'albero. In particolare, ciascun nodo presenta i seguenti attributi:

- `name`, identifica l'item rappresentato dal nodo;
- `support`, memorizza il supporto associato al nodo;
- `parent`, contiene un riferimento al nodo padre;
- `children`, un dizionario contenente i nodi figli del nodo corrente, indicizzati in base al nome dell'item.

L'implementazione della costruzione dell'albero è suddivisa nelle due seguenti funzioni: `find_1_itemsets_fpgrowth(transactions.counts, minsup)` per l'estrazione

degli itemset frequenti di dimensione 1 e `build_fp_tree(transactions_counts, minsup)` per la costruzione vera e propria dell'albero. La prima funzione viene utilizzata per individuare gli item frequenti di dimensione 1 e per calcolare il supporto di tutti gli item presenti nel dataset. Per ogni transazione viene considerato ciascun item in essa contenuto e il relativo supporto viene incrementato in base al conteggio associato alla transazione. Al termine di questa fase vengono mantenuti esclusivamente gli item il cui supporto è maggiore o uguale alla soglia minima `minsup`. La funzione restituisce quindi la lista degli item frequenti e un dizionario contenente il supporto di ciascun item. La seconda funzione utilizza queste informazioni per costruire l'FP-Tree. Gli item frequenti vengono inizialmente ordinati in ordine decrescente rispetto al loro supporto. In caso di supporto uguale viene utilizzato il nome dell'item come criterio secondario di ordinamento. Questo ordinamento consente alle transazioni che condividono gli item più frequenti di condividere anche i prefissi dei relativi percorsi nell'albero, permettendo così una rappresentazione compatta delle transazioni. Successivamente viene costruita la header table. Essa è rappresentata da un dizionario che associa ogni item frequente alla lista dei nodi dell'FP-Tree che rappresentano quell'item. La header table può quindi essere considerata come un indice che permette di individuare rapidamente tutte le occorrenze di uno specifico item nell'albero. Viene quindi inizializzata la radice dell'FP-Tree, che non rappresenta alcun item. Per ogni transazione vengono eliminati gli item non frequenti e gli item rimanenti vengono ordinati secondo lo stesso criterio utilizzato in precedenza. La transazione filtrata e ordinata viene successivamente inserita nell'albero a partire dalla radice. Per ogni item della transazione viene verificato se esiste già, tra i figli del nodo corrente, un nodo associato allo stesso item. Se il nodo esiste, esso viene riutilizzato e il suo supporto viene incrementato del conteggio associato alla transazione. In caso contrario viene creato un nuovo nodo, collegato al proprio padre e aggiunto sia alla struttura `children` del padre sia alla lista corrispondente nella header table. Il riutilizzo dei nodi permette a transazioni che condividono lo stesso prefisso di condividere lo stesso percorso nell'albero. L'FP-Tree costituisce quindi una rappresentazione compatta del database delle transazioni, nella quale le informazioni relative agli item frequenti vengono aggregate attraverso percorsi comuni.

3.3.2 Fase 2: estrazione degli itemset frequenti

Una volta costruito l'FP-Tree, la seconda fase dell'algoritmo consiste nell'estrazione degli itemset frequenti con le funzioni `build_conditional_pattern_base(header_table, item_name)` e `mine_fp_tree(transactions_counts, minsup, max_len, prefix_items)`. La prima funzione costruisce la conditional pattern base relativa a uno specifico item. Per ogni nodo associato all'item considerato nella header table, si risale l'albero attraverso i riferimenti al nodo padre, partendo dal padre del nodo corrente e procedendo verso la radice. Durante questa risalita vengono raccolti i nomi degli item presenti nei nodi attraversati. La radice viene esclusa poiché non rappresenta alcun item. Il percorso ottenuto rappresenta quindi il prefisso che conduce al nodo dell'item considerato. Se il percorso contiene almeno un

item, esso viene inserito nella conditional pattern base insieme al supporto del nodo di partenza. L'estrazione vera e propria degli itemset viene effettuata dalla seconda funzione. La funzione costruisce innanzitutto un nuovo FP-Tree a partire dalle transazioni ricevute. Gli item presenti nella header table vengono quindi ordinati in ordine crescente rispetto al loro supporto. Per ogni item viene costruito un nuovo itemset, indicato come `current_itemset`, ottenuto aggiungendo l'item corrente agli eventuali item già presenti nel prefisso `prefix_items`. L'itemset così ottenuto viene aggiunto all'insieme dei pattern individuati. Se la lunghezza dell'itemset non ha ancora raggiunto il valore massimo `max_len`, viene costruita la conditional pattern base relativa all'item corrente. Quest'ultima viene utilizzata come input per una nuova chiamata ricorsiva di `mine_fp_tree`, passando come nuovo prefisso l'itemset appena costruito. La procedura viene quindi ripetuta ricorsivamente, consentendo di individuare itemset di dimensione progressivamente maggiore. La ricorsione termina quando viene raggiunta la lunghezza massima specificata da `max_len` oppure quando non è più possibile costruire ulteriori pattern frequenti. La funzione `fpgrowth_mining(dataset, minsup, max_len)` rappresenta il punto di ingresso dell'implementazione. In sintesi, l'implementazione segue quindi il principio fondamentale di FP-Growth: il database viene dapprima compresso attraverso la costruzione dell'FP-Tree e, successivamente, l'estrazione degli itemset frequenti viene effettuata analizzando ricorsivamente le conditional pattern base associate ai diversi item. In questo modo non è necessario generare esplicitamente tutti i possibili itemset candidati, riducendo il numero di operazioni necessarie per l'estrazione dei pattern frequenti.

4 Analisi dei risultati

In questa sezione vengono analizzati i risultati ottenuti dal confronto tra i tre algoritmi considerati: CLA, Apriori e FP-Growth. Il confronto è stato effettuato su sei dataset diversi: Mushroom, Retail, Pumsb, Kosarak, Chess e Connect. Per ciascun dataset sono stati considerati quattro valori della soglia minima di supporto:

$$minsup \in \{0.02, 0.04, 0.06, 0.08\}$$

Per ogni configurazione sono stati misurati i tempi di esecuzione dei tre algoritmi e sono stati confrontati gli itemset frequenti trovati. In tutti i casi gli algoritmi hanno restituito gli stessi itemset. Questo significa che i tre algoritmi producono esattamente gli stessi risultati e che le differenze osservate riguardano solamente il tempo necessario per ottenere tali risultati.

4.1 Dataset Mushroom

Nel dataset Mushroom si osservano differenze molto significative tra i tre algoritmi, soprattutto tra FP-Growth e gli altri due metodi.

- Per $\text{minsup} = 0.02$ vengono trovati 22513 itemset frequenti. CLA impiega 12.0354 secondi, Apriori 75.6909 secondi, mentre FP-Growth impiega solamente 0.7844 secondi. FP-Growth risulta quindi nettamente più veloce rispetto agli altri due algoritmi.
- Per $\text{minsup} = 0.04$ il numero di itemset frequenti diminuisce a 13671. I tempi di esecuzione sono 14.5144 secondi per CLA, 45.4468 secondi per Apriori e 1.1182 secondi per FP-Growth. Anche in questo caso FP-Growth presenta il tempo di esecuzione più basso.
- Per $\text{minsup} = 0.06$ vengono trovati 9705 itemset frequenti. CLA impiega 10.1515 secondi, Apriori 32.9716 secondi e FP-Growth 0.4384 secondi. Anche in questa configurazione il vantaggio di FP-Growth è molto evidente.
- Per $\text{minsup} = 0.08$ vengono trovati 6605 itemset frequenti. I tempi sono 7.2364 secondi per CLA, 22.5916 secondi per Apriori e 0.3106 secondi per FP-Growth.

Dal grafico relativo al dataset Mushroom si può quindi osservare che FP-Growth è sempre l'algoritmo più veloce. Apriori è invece sempre il più lento, mentre CLA si trova in una posizione intermedia. Un aspetto interessante è la diminuzione del numero di itemset frequenti all'aumentare di minsup . Si passa infatti da 22513 itemset con $\text{minsup} = 0.02$ a 6605 itemset con $\text{minsup} = 0.08$. L'aumento della soglia minima di supporto permette quindi di eliminare un numero maggiore di itemset non frequenti e, in generale, comporta una riduzione del tempo di esecuzione. Il comportamento di FP-Growth è particolarmente positivo su questo dataset. Il numero elevato di itemset frequenti rende vantaggiosa la rappresentazione compatta delle transazioni attraverso l'FP-Tree, permettendo di ottenere un tempo di esecuzione inferiore rispetto ad Apriori e CLA.

Dal confronto con i risultati riportati nel paper originale sul dataset Mushroom emergono diverse analogie e alcune differenze rilevanti. In entrambi i casi si osserva una stessa tendenza: all'aumentare della soglia di supporto minimo, i tempi di esecuzione diminuiscono progressivamente per tutti gli algoritmi a causa della riduzione dello spazio di ricerca degli itemset frequenti. Nel lavoro di riferimento, l'algoritmo basato su CLA si attesta come il più veloce in assoluto, superando SABMA e PP-Tree con un tempo di esecuzione compreso tra circa 4 e 9 secondi. Nell'implementazione proposta, CLA mantiene un andamento molto simile e coerente con la letteratura, registrando tempi tra 7 e 15 secondi, ma risulta essere più lento di FP-Growth, i cui tempi rimangono costantemente inferiori ai 2 secondi. Tale divergenza è sicuramente da ricondurre alla diversa implementazione dei due metodi e alla differente modalità di esecuzione. L'introduzione di Apriori nel secondo grafico evidenzia infine tempi significativamente superiori, fino a oltre 75 secondi con supporto pari a 0.02, confermando l'efficacia generale delle metodologie basate su CLA ed FP-Growth nel contenere la complessità computazionale.

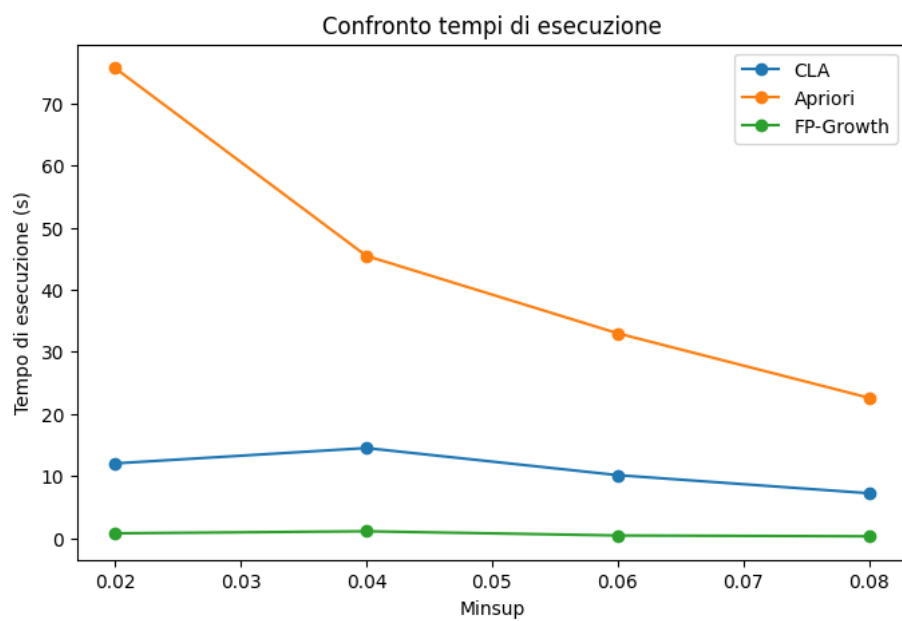


Figure 1: Tempi di esecuzione sul dataset Mushroom

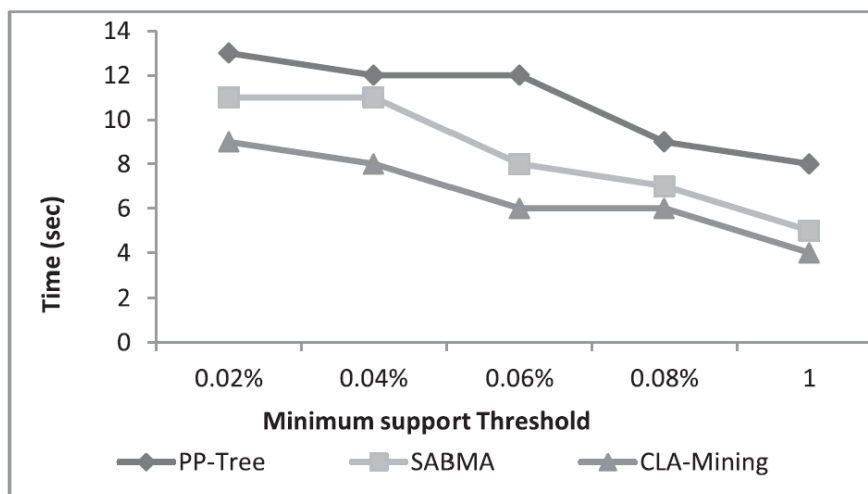


Fig. 14. Dataset mushroom.

Figure 2: Tempi di esecuzione sul dataset Mushroom nell'articolo di riferimento

4.2 Dataset Retail

Il comportamento del dataset Retail è diverso rispetto a quello osservato per Mushroom. In questo caso il numero di itemset frequenti è molto più basso e le differenze tra i tre algoritmi sono meno marcate.

- Per $\text{minsup} = 0.02$ vengono trovati 54 itemset frequenti. CLA impiega 0.3353 secondi, Apriori 3.6296 secondi e FP-Growth 0.7293 secondi. In questa configurazione CLA è quindi l'algoritmo più veloce, mentre Apriori è il più lento.
- Per $\text{minsup} = 0.04$ vengono trovati 18 itemset. I tempi sono 0.3969 secondi per CLA, 1.2865 secondi per Apriori e 0.5386 secondi per FP-Growth. Anche in questo caso CLA risulta più veloce di FP-Growth.
- Per $\text{minsup} = 0.06$ vengono trovati 15 itemset. In questa configurazione FP-Growth diventa l'algoritmo più veloce, con un tempo di 0.2983 secondi. CLA impiega 0.4019 secondi, mentre Apriori impiega 0.5707 secondi.
- Per $\text{minsup} = 0.08$ vengono trovati solamente 13 itemset. CLA impiega 0.2262 secondi, FP-Growth 0.2915 secondi e Apriori 0.5136 secondi. CLA torna quindi ad essere l'algoritmo più veloce.

Il dataset Retail mostra quindi che FP-Growth non è necessariamente il metodo più veloce in ogni situazione. Quando il numero di itemset frequenti è molto basso, il vantaggio derivante dalla rappresentazione tramite FP-Tree può essere ridotto. Il costo necessario per costruire l'albero e per effettuare l'estrazione ricorsiva dei pattern può infatti diventare rilevante rispetto al tempo complessivo di esecuzione. In tutte le configurazioni Apriori risulta comunque più lento rispetto agli altri due algoritmi. Tuttavia, le differenze sono molto più contenute rispetto a quelle osservate nel dataset Mushroom.

Dal confronto tra i due grafici relativi al dataset Retail si osserva che per entrambi si conferma una tendenza generale alla diminuzione dei tempi di esecuzione all'aumentare della soglia di supporto minimo, dovuta alla riduzione dello spazio di ricerca. Nel lavoro originale l'algoritmo CLA si conferma l'algoritmo più veloce in assoluto, superando SABMA e PP-Tree con tempi compresi tra circa 13 e 4 secondi. Nell'implementazione proposta, l'algoritmo CLA mostra tempi di esecuzione più rapidi (tra circa 0.2 e 0.4 secondi), risultando quasi per tutte le soglie di minsup il metodo più veloce. Rispetto al grafico del paper dove PP-Tree mostra tempi più elevati compresi tra 13 e 21 secondi, la versione sequenziale di FP-Growth si attesta su tempi estremamente contenuti (sotto i 0.8 secondi), sovrapponendosi alle prestazioni dell'algoritmo CLA per valori di supporto maggiori di 0.05. L'inclusione di Apriori evidenzia tempi marcatamente più alti a supporto ridotto, arrivando a circa 3.6 secondi con valore 0.02, per poi decrescere rapidamente e allinearsi attorno al mezzo secondo. Nel complesso, l'implementazione proposta dimostra un'elevata efficienza computazionale sul dataset Retail, confermando la superiorità di CLA ed FP-Growth rispetto ad approcci tradizionali come Apriori.

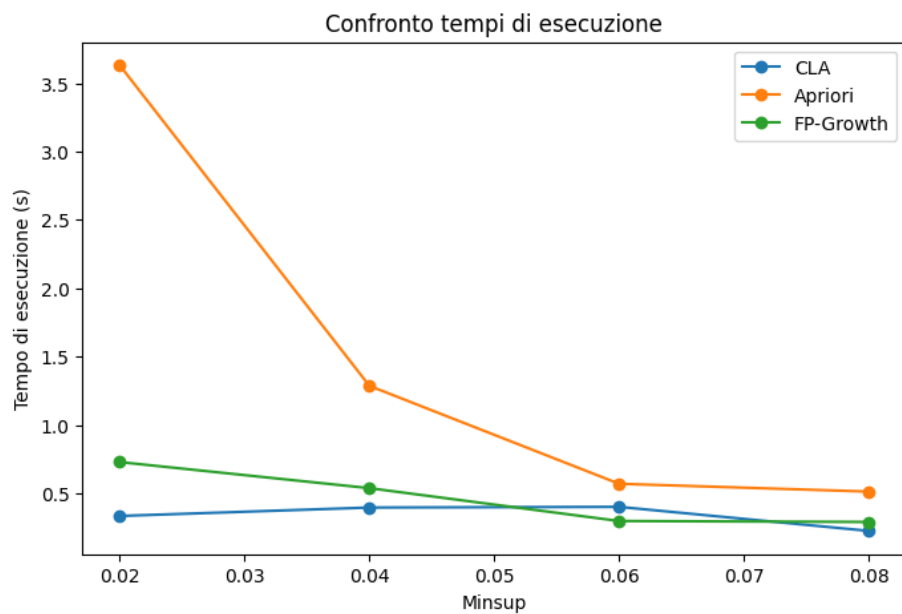


Figure 3: Tempi di esecuzione sul dataset Retail

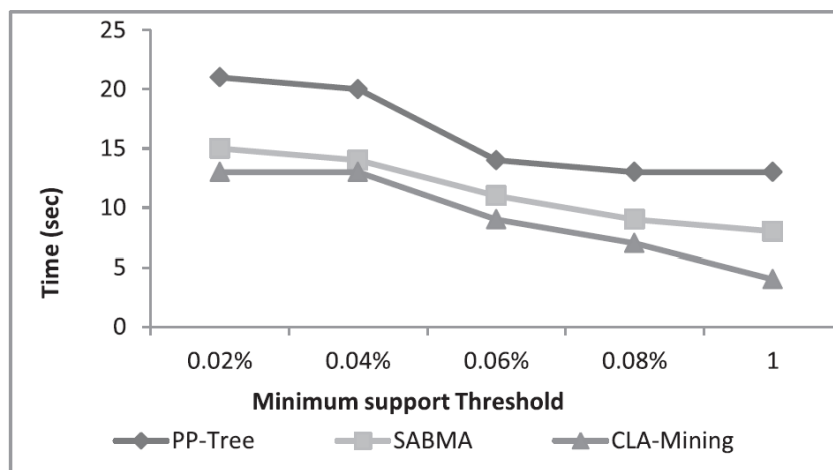


Fig. 15. Dataset retail.

Figure 4: Tempi di esecuzione sul dataset Retail nell'articolo di riferimento

4.3 Dataset Pumsb

Il dataset Pumsb presenta i tempi di esecuzione più elevati tra tutti i dataset considerati. Questo comportamento è legato soprattutto all'elevato numero di itemset frequenti individuati.

- Per $\text{minsup} = 0.02$ vengono trovati 455351 itemset frequenti. CLA impiega 3448.6976 secondi, Apriori 10154.0501 secondi e FP-Growth 600.8944 secondi. I tempi corrispondono rispettivamente a circa 57.5 minuti, 169.2 minuti e 10 minuti. In questa configurazione FP-Growth è quindi nettamente più veloce rispetto agli altri due algoritmi. La differenza è particolarmente evidente nel confronto con Apriori.
- Per $\text{minsup} = 0.04$ il numero di itemset frequenti diminuisce a 276889. CLA impiega 3157.18 secondi, Apriori 6666.2923 secondi e FP-Growth 456.5612 secondi.
- Per $\text{minsup} = 0.06$ vengono trovati 208506 itemset frequenti. I tempi sono 2728.0567 secondi per CLA, 4994.3771 secondi per Apriori e 359.0799 secondi per FP-Growth.
- Per $\text{minsup} = 0.08$ vengono trovati 167297 itemset frequenti. CLA impiega 2422.9886 secondi, Apriori 4141.1949 secondi e FP-Growth 291.4345 secondi.

Anche osservando il grafico relativo a Pumsb si nota quindi che FP-Growth è sempre il metodo più veloce. Apriori è sempre il più lento, mentre CLA presenta tempi intermedi. Il risultato più importante per questo dataset è la grande differenza tra i tempi di esecuzione. Anche aumentando minsup fino a 0.08, vengono ancora trovati 167297 itemset frequenti. Di conseguenza, il problema rimane molto costoso dal punto di vista computazionale. In questo caso il vantaggio della struttura FP-Tree risulta particolarmente evidente. FP-Growth riesce, infatti, a gestire un numero molto elevato di itemset frequenti con tempi significativamente inferiori rispetto agli altri due algoritmi.

Dal confronto tra i due grafici relativi al dataset Pumsb si osserva nuovamente che i tempi di esecuzione diminuiscono all'aumentare della soglia di supporto minimo. Nel grafico del paper originale l'algoritmo CLA si conferma l'algoritmo più rapido con tempi compresi tra circa 21 e 8 secondi, superando SABMA e PP-Tree le cui prestazioni si attestano rispettivamente tra 29 e 11 secondi e tra 34 e 19 secondi. Nell'implementazione proposta si registra una scala dei tempi significativamente più elevata a causa della densità del dataset Pumsb. In questo caso, la versione sequenziale di FP-Growth quella più efficiente mantenendo tempi compresi tra circa 600 e 300 secondi, mentre l'implementazione di CLA si posiziona al secondo posto con tempi che variano da circa 3500 a 2400 secondi. Come per gli altri risultati, Apriori risulta l'algoritmo più lento, impiegando fino a oltre 10000 secondi per supporto 0.02 prima di scendere progressivamente verso i 4000 secondi a supporto 0.08. Nonostante la grande differenza nei tempi, i risultati relativi a CLA e agli approcci di tipo Apriori rimangono

coerenti, evidenziando il vantaggio dell'approccio basato sui cellular learning automata.

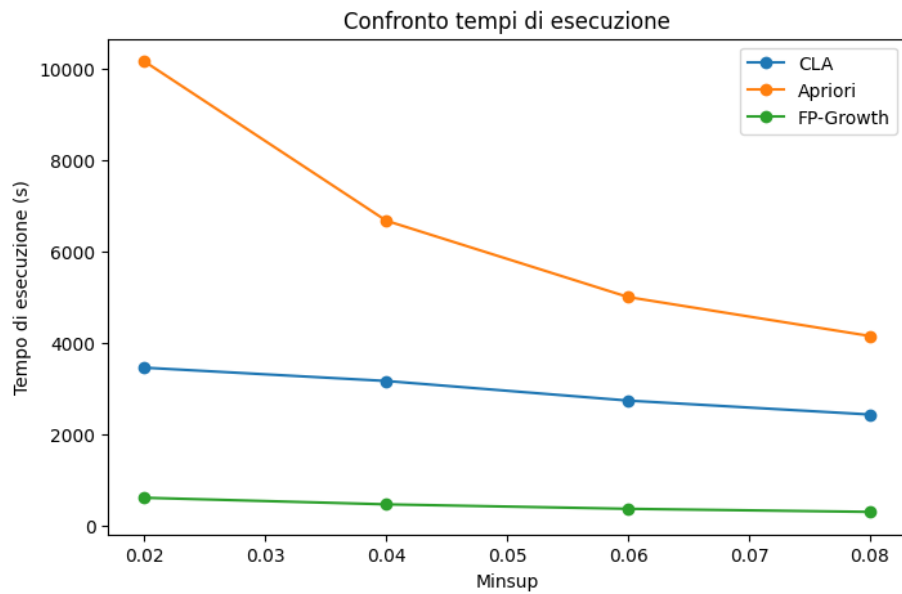


Figure 5: Tempi di esecuzione sul dataset Pumsb

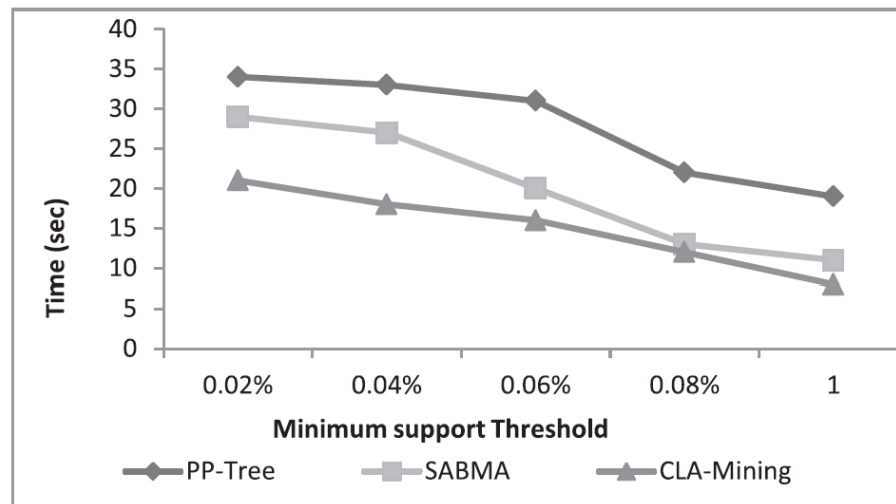


Fig. 16. Dataset pumsb.

Figure 6: Tempi di esecuzione sul dataset Pumsb nell'articolo di riferimento

4.4 Dataset Kosarak

Il dataset Kosarak presenta un comportamento differente rispetto a Mushroom e Pumsb. FP-Growth risulta il più veloce quando minsup è pari a 0.02, mentre per valori maggiori della soglia CLA presenta tempi inferiori.

- Per minsup = 0.02 vengono trovati 106 itemset frequenti. CLA impiega 8.9369 secondi, Apriori 97.9181 secondi e FP-Growth 6.9927 secondi. In questo caso FP-Growth è quindi l'algoritmo più veloce.
- Per minsup = 0.04 il numero di itemset frequenti diminuisce a 41. CLA impiega 3.1269 secondi, Apriori 25.4608 secondi e FP-Growth 4.6465 secondi. In questa configurazione CLA diventa il metodo più veloce.
- Per minsup = 0.06 vengono trovati 24 itemset. I tempi sono 3.8919 secondi per CLA, 15.5135 secondi per Apriori e 4.5499 secondi per FP-Growth. Anche in questo caso CLA risulta leggermente più veloce di FP-Growth.
- Per minsup = 0.08 vengono trovati 14 itemset frequenti. CLA impiega 2.6784 secondi, Apriori 8.5597 secondi e FP-Growth 4.2968 secondi.

Il comportamento di Kosarak mostra quindi che il metodo più veloce può dipendere sia dal dataset sia dal valore della soglia minima di supporto. Per minsup = 0.02 FP-Growth presenta il tempo migliore, mentre per i valori successivi CLA risulta più veloce. Una possibile spiegazione è che, quando il numero di itemset frequenti è ridotto, il vantaggio derivante dalla costruzione dell'FP-Tree può diminuire. In queste situazioni il costo aggiuntivo necessario per costruire l'albero e analizzare ricorsivamente le conditional pattern base può rendere FP-Growth meno conveniente rispetto a CLA. Anche in questo dataset Apriori presenta sempre tempi superiori rispetto agli altri due algoritmi.

Come osservato per i dataset precedenti, dal confronto tra i due grafici relativi al dataset Kosarak emerge una generale diminuzione dei tempi di esecuzione all'aumentare della soglia di supporto minimo. Nel grafico relativo all'articolo di riferimento, i tempi risultano più alti per tutte le configurazioni, rimanendo in un intervallo compreso tra circa 65 e 120 secondi. Anche in questo caso l'algoritmo CLA registra i tempi più bassi, superando SABMA e PP-Tree. Nell'implementazione proposta, sia CLA che FP-Growth mostrano invece un'efficienza superiore, stabilizzandosi entrambi su tempi compresi tra circa 3 e 9 secondi lungo l'intero intervallo di soglie di supporto. In quest'ultimo grafico, le curve di CLA e FP-Growth risultano pressoché sovrapposte, con CLA che riesce a superare leggermente FP-Growth a partire da un supporto pari a 0.04. L'algoritmo Apriori mostra, invece, un tempo iniziale elevato, toccando quasi i 100 secondi per il supporto 0.02, per poi calare sotto i 30 secondi per il supporto 0.04 e allinearsi attorno ai 10 secondi per valori superiori. Nel complesso, pur con una scala di tempi diversa dovuta all'ambiente hardware e alle differenze implementative, la nuova sperimentazione riconferma l'elevata efficienza di CLA, evidenziandone la capacità di competere direttamente con FP-Growth e di staccare nettamente l'approccio Apriori.

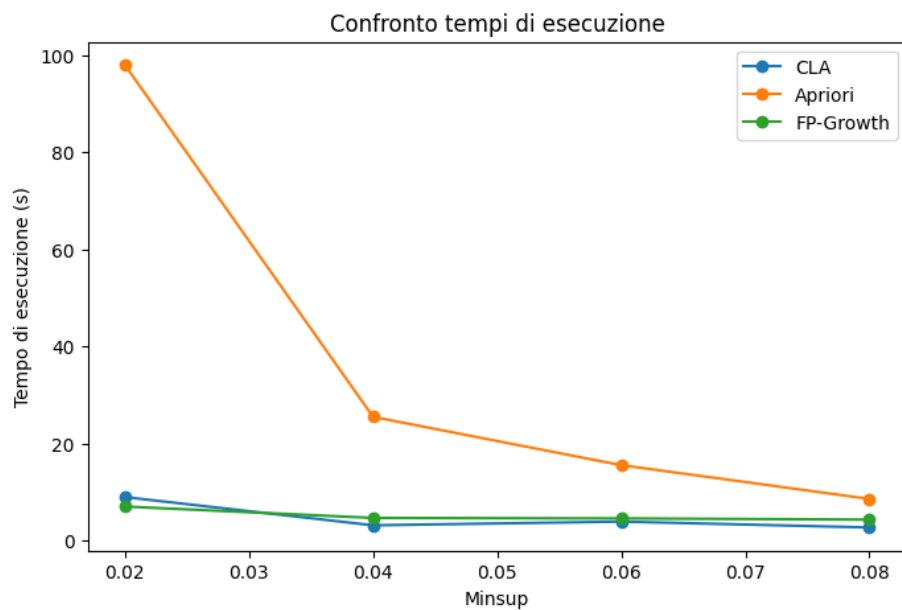


Figure 7: Tempi di esecuzione sul dataset Kosarak

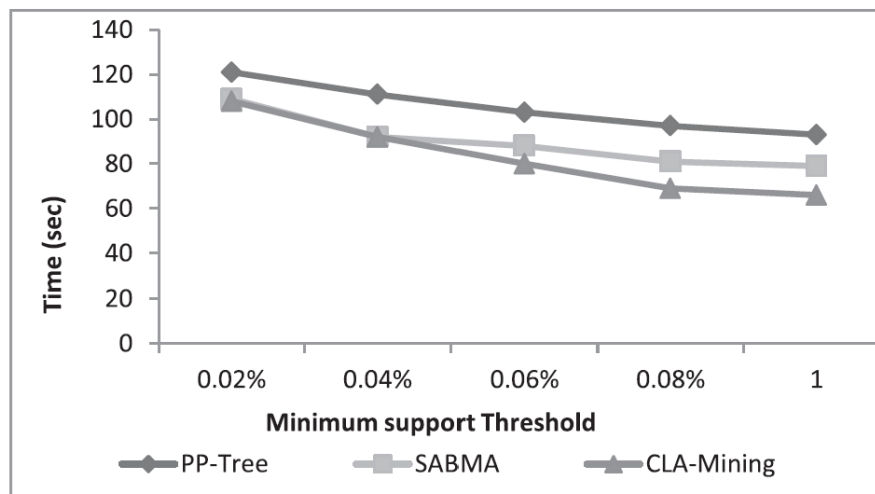


Fig. 18. Dataset kosarak.

Figure 8: Tempi di esecuzione sul dataset Kosarak nell'articolo di riferimento

4.5 Dataset Chess

Nel dataset Chess si osservano differenze significative tra i tre algoritmi, soprattutto per quanto riguarda i tempi di esecuzione. Anche in questo caso, tutti gli algoritmi individuano esattamente gli stessi itemset frequenti per ciascun valore della soglia minima di supporto.

- Per $\text{minsup} = 0.02$ vengono trovati 39480 itemset frequenti. CLA impiega 14.7158 secondi, Apriori 47.6061 secondi, mentre FP-Growth impiega 6.0237 secondi. FP-Growth presenta quindi il tempo di esecuzione più basso, seguito da CLA, mentre Apriori risulta il più lento.
- Per $\text{minsup} = 0.04$ il numero di itemset frequenti diminuisce a 32572. I tempi di esecuzione sono 15.9939 secondi per CLA, 37.7941 secondi per Apriori e 7.3652 secondi per FP-Growth. Anche in questa configurazione FP-Growth risulta il più veloce.
- Per $\text{minsup} = 0.06$ vengono trovati 28113 itemset frequenti. CLA impiega 14.1708 secondi, Apriori 35.1054 secondi e FP-Growth 5.6184 secondi. FP-Growth mantiene quindi il tempo di esecuzione più basso, mentre Apriori rimane il metodo più lento.
- Per $\text{minsup} = 0.08$ vengono trovati 25297 itemset frequenti. I tempi sono 14.6485 secondi per CLA, 30.0721 secondi per Apriori e 6.8111 secondi per FP-Growth.

Dal grafico relativo al dataset Chess si può quindi osservare che FP-Growth è sempre l'algoritmo più veloce tra i tre metodi considerati. Apriori risulta invece sempre il più lento, mentre CLA presenta tempi di esecuzione intermedi. La differenza tra gli algoritmi è particolarmente evidente nel confronto con Apriori: per $\text{minsup} = 0.02$, ad esempio, Apriori richiede 47.6061 secondi, mentre FP-Growth ne richiede solamente 6.0237.

Il comportamento di FP-Growth risulta particolarmente interessante considerando la struttura relativamente densa del dataset Chess. La rappresentazione compatta delle transazioni mediante l'FP-Tree consente di evitare parte della generazione esplicita dei candidati tipica dell'approccio Apriori, permettendo di ottenere tempi di esecuzione sensibilmente inferiori. CLA presenta invece prestazioni intermedie, con tempi relativamente stabili al variare della soglia di supporto.

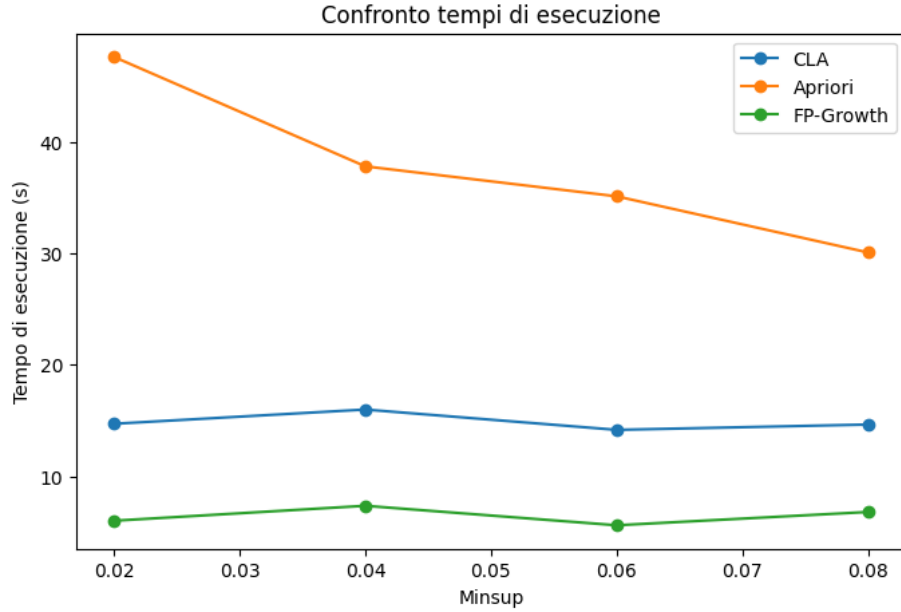


Figure 9: Tempi di esecuzione sul dataset Chess

4.6 Dataset Connect

Nel dataset Connect le differenze tra i tre algoritmi risultano ancora più evidenti rispetto al dataset Chess. In particolare, FP-Growth presenta tempi di esecuzione sensibilmente inferiori rispetto sia a CLA sia ad Apriori, mentre quest'ultimo risulta il metodo con il tempo di esecuzione più elevato in tutte le configurazioni considerate.

- Per $\text{minsup} = 0.02$ vengono trovati 74962 itemset frequenti. CLA impiega 593.7043 secondi, Apriori 1670.1731 secondi, mentre FP-Growth impiega 42.9977 secondi. FP-Growth presenta quindi un tempo di esecuzione nettamente inferiore rispetto agli altri due algoritmi.
- Per $\text{minsup} = 0.04$ il numero di itemset frequenti diminuisce a 56682. I tempi di esecuzione sono 590.0453 secondi per CLA, 1360.3039 secondi per Apriori e 29.7740 secondi per FP-Growth. Anche in questo caso FP-Growth risulta sensibilmente più veloce.
- Per $\text{minsup} = 0.06$ vengono trovati 45210 itemset frequenti. CLA impiega 572.8340 secondi, Apriori 1056.5803 secondi e FP-Growth 23.6377 secondi. La differenza tra FP-Growth e gli altri due algoritmi rimane quindi molto evidente.
- Per $\text{minsup} = 0.08$ vengono trovati 38459 itemset frequenti. I tempi sono 515.9660 secondi per CLA, 924.2966 secondi per Apriori e 16.9606 secondi

per FP-Growth.

Dal grafico relativo al dataset Connect si può osservare che FP-Growth è il metodo con il tempo di esecuzione più basso per tutti i valori di minsup considerati, mentre Apriori presenta sistematicamente il tempo più elevato. CLA si colloca tra i due, con tempi significativamente inferiori rispetto ad Apriori ma ancora molto superiori rispetto a FP-Growth.

La differenza risulta particolarmente evidente per $\text{minsup} = 0.02$, dove FP-Growth completa l'esecuzione in 42.9977 secondi, mentre CLA richiede 593.7043 secondi e Apriori 1670.1731 secondi. Analogamente, per $\text{minsup} = 0.08$ FP-Growth impiega 16.9606 secondi, contro 515.9660 secondi di CLA e 924.2966 secondi di Apriori. Il vantaggio temporale di FP-Growth risulta quindi particolarmente significativo sul dataset Connect.

Il comportamento osservato sul dataset Connect può essere messo in relazione con le caratteristiche del dataset, che presenta un numero elevato di transazioni e una struttura relativamente densa. In queste condizioni, la possibilità di rappresentare in maniera compatta le informazioni contenute nelle transazioni attraverso l'FP-Tree risulta particolarmente vantaggiosa. Apriori, al contrario, deve affrontare il costo associato alla generazione e alla verifica dei candidati, con un conseguente aumento significativo del tempo di esecuzione. CLA presenta prestazioni intermedie, ma anche in questo caso il divario rispetto a FP-Growth rimane molto elevato.

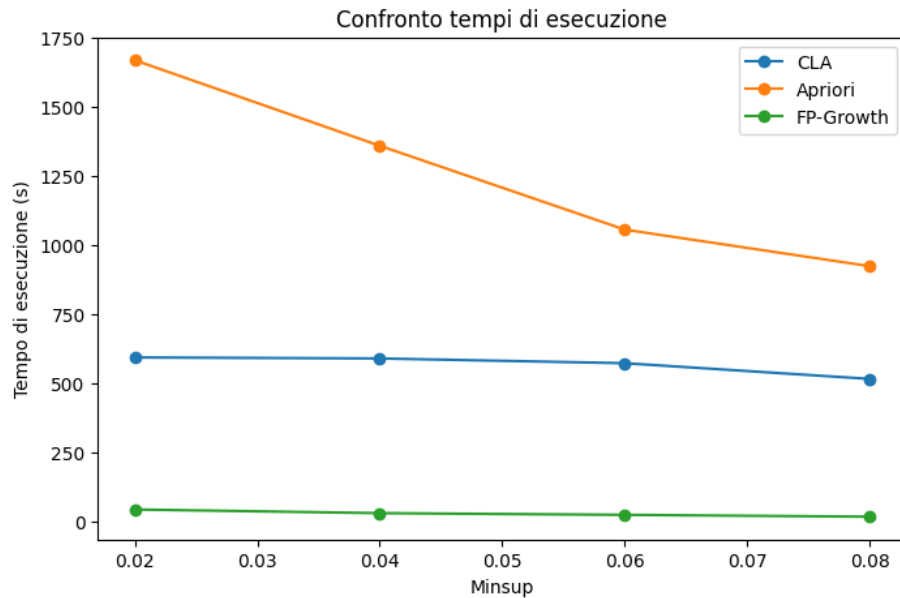


Figure 10: Tempi di esecuzione sul dataset Connect

4.7 Confronto complessivo

Considerando complessivamente i risultati ottenuti sui sei dataset, emerge che le prestazioni dei tre algoritmi dipendono in maniera significativa dalle caratteristiche del dataset e dalla soglia minima di supporto considerata. In particolare, il numero di item distinti, la lunghezza media delle transazioni, la densità della rappresentazione e il numero di itemset frequenti da estrarre influenzano in modo rilevante il tempo necessario per completare l'esecuzione.

FP-Growth presenta prestazioni particolarmente favorevoli sui dataset caratterizzati da un numero elevato di itemset frequenti. Questo comportamento è evidente nei dataset Mushroom, Pumsb, Chess e Connect. Nel caso di Mushroom, FP-Growth è sempre il metodo più veloce e presenta tempi sensibilmente inferiori rispetto a CLA e Apriori. Anche su Pumsb il vantaggio di FP-Growth risulta particolarmente significativo, soprattutto considerando l'elevato numero di itemset frequenti generati alle soglie di supporto più basse. Un comportamento analogo si osserva nei dataset Chess e Connect. Chess presenta 39480 itemset frequenti per $\text{minsup} = 0.02$, che si riducono a 25297 per $\text{minsup} = 0.08$. In tutte le configurazioni FP-Growth risulta il metodo più veloce, con tempi compresi tra 5.6184 e 7.3652 secondi. CLA presenta tempi compresi tra 14.1708 e 15.9939 secondi, mentre Apriori richiede tra 30.0721 e 47.6061 secondi. Il vantaggio di FP-Growth risulta quindi evidente anche su un dataset relativamente denso e con un numero di item distinti contenuto. Il comportamento è ancora più evidente nel dataset Connect. In questo caso il numero di itemset frequenti passa da 74962 per $\text{minsup} = 0.02$ a 38459 per $\text{minsup} = 0.08$. FP-Growth è il più veloce in tutte le configurazioni, con tempi che diminuiscono da 42.9977 a 16.9606 secondi. CLA presenta invece tempi compresi tra 515.9660 e 593.7043 secondi, mentre Apriori passa da 1670.1731 a 924.2966 secondi. La differenza tra FP-Growth e gli altri due algoritmi è quindi particolarmente elevata su questo dataset.

I risultati relativi a Retail e Kosarak mostrano invece che il rapporto tra numero di itemset frequenti e tempo di esecuzione non è sufficiente a spiegare completamente le prestazioni degli algoritmi. Retail presenta un numero molto elevato di item distinti, ma transazioni mediamente brevi e un numero relativamente contenuto di itemset frequenti. In questo scenario CLA risulta in alcune configurazioni più veloce di FP-Growth. Kosarak presenta una struttura ancora più sparsa, con un numero molto elevato di transazioni e di item distinti. In questo caso FP-Growth risulta il più veloce per $\text{minsup} = 0.02$, mentre CLA risulta più veloce alle soglie successive. Questi risultati evidenziano come la struttura del dataset possa influenzare il comportamento degli algoritmi anche quando il numero di itemset frequenti è simile.

Per quanto riguarda Apriori, dai risultati emerge un comportamento generalmente meno favorevole in termini di tempo di esecuzione. L'algoritmo risulta infatti il più lento in numerose configurazioni, in particolare nei dataset caratterizzati da un numero elevato di itemset frequenti. La differenza è particolarmente evidente nei dataset Chess e Connect. Sul dataset Connect, ad esempio, Apriori richiede 1670.1731 secondi con $\text{minsup} = 0.02$, a fronte dei 593.7043 secondi di

CLA e dei soli 42.9977 secondi di FP-Growth. Anche aumentando la soglia a 0.08, Apriori rimane il metodo più lento, con un tempo di 924.2966 secondi.

Un altro risultato importante riguarda l'effetto della soglia minima di supporto. In tutti i dataset considerati, l'aumento di minsup determina una riduzione del numero di itemset frequenti. Questo fenomeno è particolarmente evidente nei dataset caratterizzati da un elevato numero di pattern. In Pumsb, ad esempio, si passa da 455351 itemset con minsup = 0.02 a 167297 con minsup = 0.08. Analogamente, Mushroom passa da 22513 a 6605 itemset, Chess da 39480 a 25297 e Connect da 74962 a 38459. La riduzione della soglia minima di supporto determina quindi uno spazio di ricerca più ampio, mentre soglie più elevate permettono di eliminare un numero maggiore di itemset non frequenti. La riduzione del numero di itemset frequenti è generalmente accompagnata da una riduzione del tempo di esecuzione, anche se il comportamento non è sempre perfettamente monotono. Questo è evidente, ad esempio, nei risultati di Chess, dove il tempo di CLA passa da 14.7158 secondi con minsup = 0.02 a 15.9939 secondi con minsup = 0.04, per poi diminuire alle soglie successive. Analogamente, il tempo di FP-Growth non diminuisce necessariamente in modo monotono. Ciò mostra che il tempo di esecuzione non dipende esclusivamente dal numero finale di itemset trovati, ma anche dalla struttura del dataset e dalle operazioni necessarie per costruire ed esplorare le strutture dati utilizzate dai diversi algoritmi.

Un elemento particolarmente importante riguarda la correttezza dei risultati prodotti. Per tutte le configurazioni sperimentali considerate il confronto tra gli insiemi di itemset restituiti dai tre algoritmi ha prodotto itemset uguali. Questo permette di attribuire le differenze osservate nei tempi di esecuzione all'efficienza computazionale dei diversi algoritmi, piuttosto che a una diversa quantità di risultati prodotti.

Nel complesso, i risultati mostrano quindi che FP-Growth presenta prestazioni particolarmente favorevoli nei dataset in cui la ricerca produce un numero elevato di itemset frequenti e, in particolare, nei dataset densi come Mushroom, Chess e Connect e nel dataset Pumsb caratterizzato da transazioni molto lunghe. Il metodo basato sui cellular learning automata risulta invece competitivo in scenari caratterizzati da una struttura più sparsa e da un numero più contenuto di pattern, come osservato per Retail e per alcune configurazioni di Kosarak. Apriori presenta generalmente tempi di esecuzione più elevati, soprattutto quando lo spazio degli itemset frequenti da esplorare diventa particolarmente ampio.

I risultati suggeriscono infine che l'efficienza dei tre algoritmi non possa essere valutata considerando esclusivamente il numero di transazioni o il numero di itemset frequenti. La combinazione tra densità, lunghezza delle transazioni, numero di item distinti e soglia minima di supporto determina infatti caratteristiche differenti dello spazio di ricerca. In questa prospettiva, l'idea proposta nel lavoro di riferimento di combinare CLA e FP-Growth può essere considerata come una possibile direzione per sfruttare caratteristiche complementari dei due approcci. Un approccio ibrido potrebbe essere oggetto di ulteriori esperimenti, verificando empiricamente se la combinazione dei due metodi possa ridurre il costo computazionale complessivo mantenendo invariato l'insieme degli itemset

frequenti individuati.

References

- [1] Sohrabi and Roshani. Frequent itemset mining using cellular learning automata. 2016.